

Objectif 1 : Sécurité avancée et politiques de sécurité sur AWS

1.1 Description de l'objectif

L'objectif de cette partie du projet est de mettre en place une stratégie de sécurité avancée sur l'infrastructure AWS afin de protéger les identités des utilisateurs, en particulier celles disposant de privilèges élevés.

Cette stratégie vise à détecter et bloquer les tentatives d'attaques ciblant les identités, à renforcer l'authentification par l'utilisation du Multi-Factor Authentication (MFA), et à restreindre les accès selon l'origine des connexions.

L'ensemble des politiques mises en place doit être testé afin de valider leur efficacité face à des scénarios d'accès légitimes et malveillants.

1.2 Protection des identités et détection des attaques

La gestion des identités est assurée par AWS Identity and Access Management (IAM). Chaque utilisateur ou rôle dispose de permissions définies par des politiques IAM.

Afin de garantir la traçabilité et la détection des comportements suspects, le service AWS CloudTrail est activé pour enregistrer toutes les actions liées aux identités (connexions, tentatives échouées, modifications de politiques).

En complément, AWS GuardDuty est utilisé pour analyser ces journaux et détecter automatiquement des activités anormales ou potentiellement malveillantes.

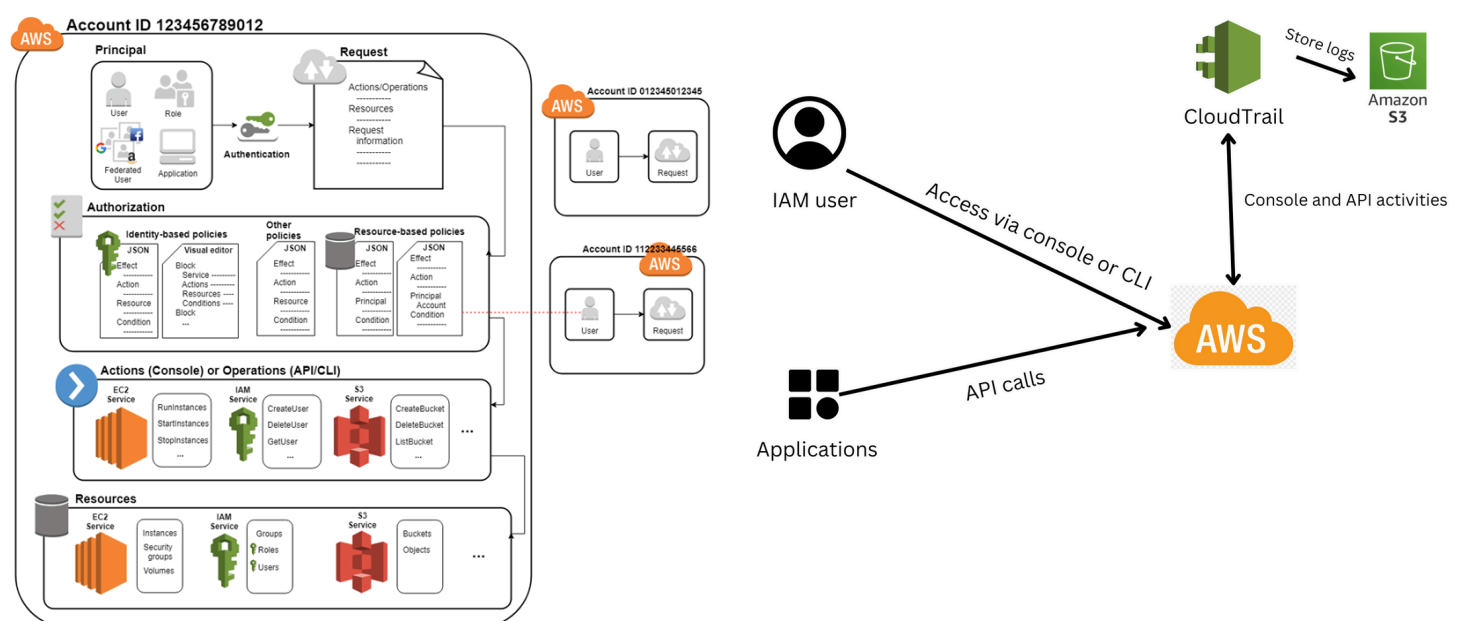


Figure 1 : Architecture de gestion des identités AWS avec IAM et journalisation des actions via CloudTrail.

Grâce à cette configuration, toute tentative d'attaque contre une identité peut être détectée et signalée en temps réel à l'équipe en charge de la sécurité.

1.3 Activation du MFA pour les comptes à privilèges élevés

Les comptes disposant de droits sensibles, tels que les administrateurs ou les officiers supérieurs de Starfleet, représentent une cible prioritaire pour les attaquants.

Pour renforcer leur sécurité, l'authentification multi-facteurs (MFA) est rendue obligatoire pour ces utilisateurs à l'aide de politiques IAM conditionnelles.

Sans validation MFA, l'accès aux ressources critiques (services de stockage, instances de calcul, bases de données) est automatiquement refusé.

Access Key ID	Created	Last Used	Last Used Service	Last Used
AKIAJSLD6X3HY7ECPORQ	2016-10-19 09:10 PDT	N/A	N/A	N/A

User Name	Rob
Password	Yes
Last Used	2016-10-21 10:16 PDT

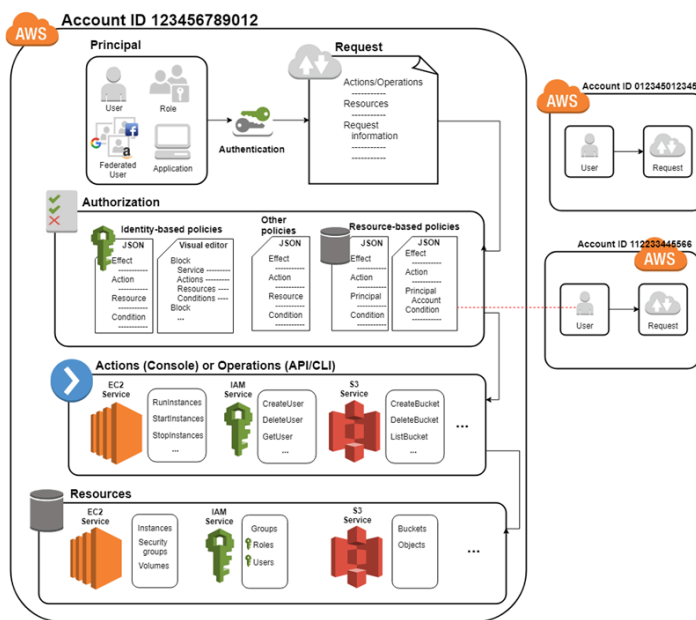


Figure 2 : Mise en place de l'authentification multi-facteurs (MFA) pour les utilisateurs IAM à privilèges élevés.

Cette mesure permet de réduire significativement le risque d'accès non autorisé, même en cas de compromission d'un mot de passe.

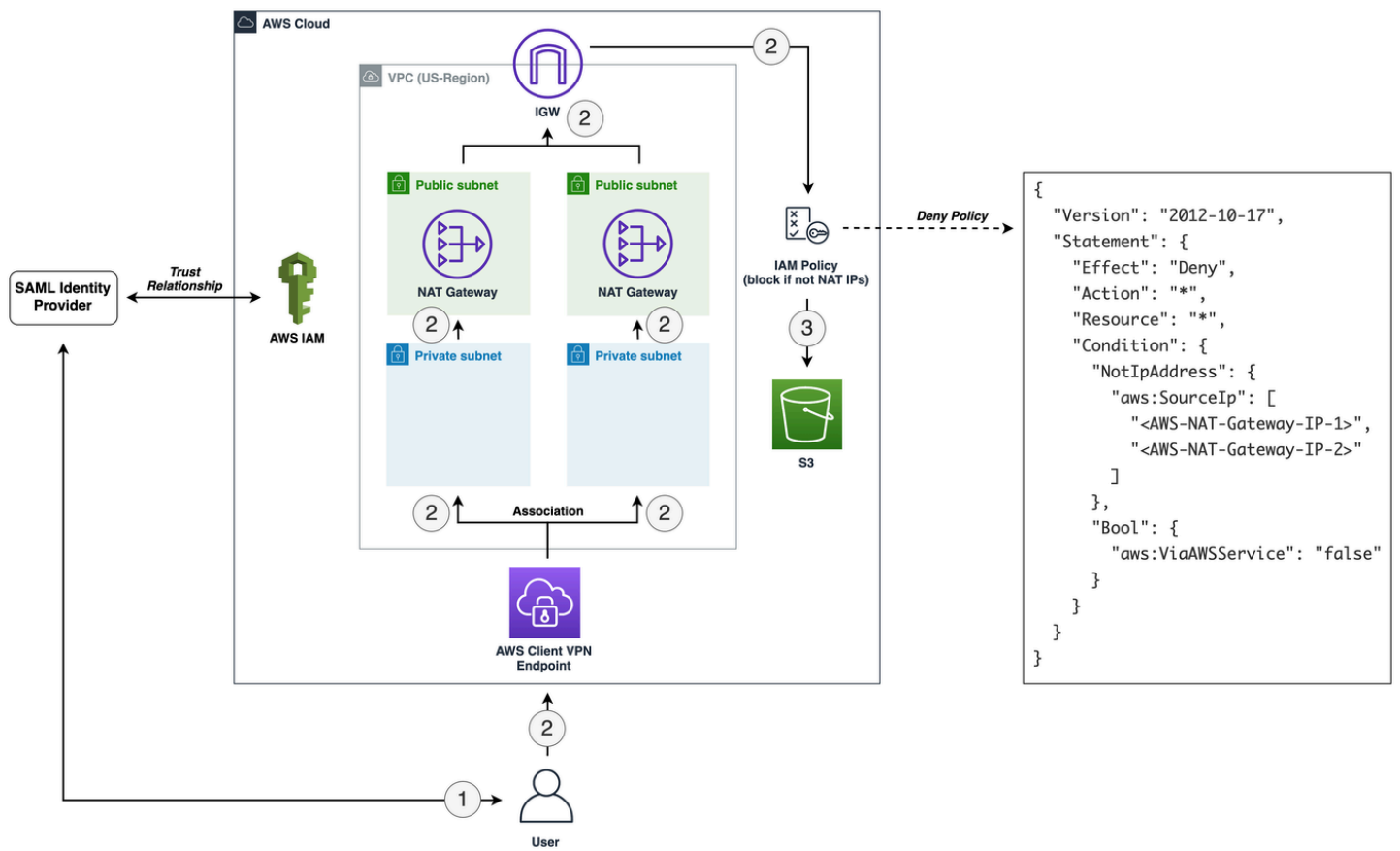
1.4 Restriction des accès selon l'origine des connexions

Afin de limiter les risques liés aux connexions provenant d'environnements non sécurisés, des **politiques IAM basées sur des conditions** sont mises en place.

Ces politiques restreignent l'accès aux ressources AWS en fonction :

- des adresses IP autorisées,
- ou des régions AWS considérées comme sûres.

Les connexions provenant de localisations non autorisées (planètes non sécurisées ou vaisseaux inconnus) sont automatiquement bloquées.





Account ID 123456789012

Principal



Request



Actions/Operations
Resources
Request information

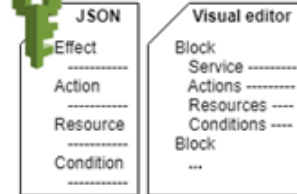
Authentication



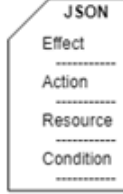
Authorization



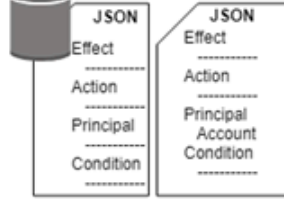
Identity-based policies



Other policies



Resource-based policies



Actions (Console) or Operations (API/CLI)



EC2 Service



RunInstances
StartInstances
StopInstances
...

IAM Service



CreateUser
DeleteUser
GetUser
...

S3 Service



CreateBucket
DeleteBucket
ListBucket
...

Resources



EC2 Service



Instances
Security groups
Volumes
...

IAM Service



Groups
Roles
Users
...

S3 Service



Buckets
Objects
...



Account ID 012345012345



Account ID 112233445566



Figure 3 : Restriction des accès AWS selon l'origine géographique ou réseau des connexions.

Ce mécanisme permet d'ajouter une couche de sécurité supplémentaire indépendante des identifiants utilisateur.

1.5 Tests et validation des politiques de sécurité

Une phase de test est réalisée afin de vérifier le bon fonctionnement des politiques mises en place.

Différents scénarios sont simulés :

- connexion depuis une localisation autorisée avec MFA actif,
- connexion depuis une localisation non autorisée,
- tentative d'accès sans MFA sur un compte à privilèges élevés.

Les alertes générées par **AWS GuardDuty** et les journaux **CloudTrail** sont analysés afin de confirmer que les comportements suspects sont correctement détectés et bloqués.

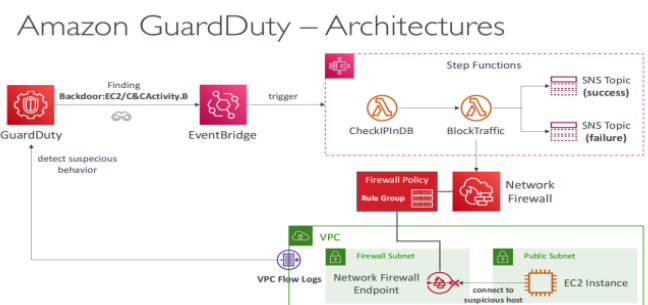
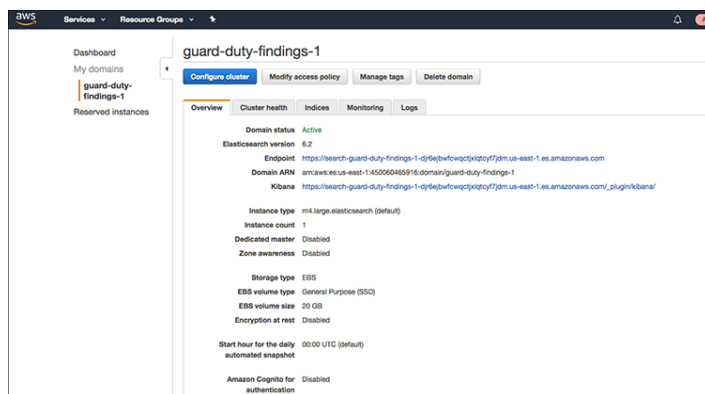


Figure 4 : Détection d'activités suspectes et alertes de sécurité générées par AWS GuardDuty.

1.6 Résultats attendus

À l'issue de cet objectif, l'infrastructure AWS bénéficie :

- d'une protection renforcée des identités,
- d'une authentification sécurisée pour les comptes sensibles,
- d'un contrôle strict des accès selon l'origine des connexions,
- d'une capacité de détection proactive des menaces,
- d'une traçabilité complète des actions critiques.

Objectif 2 : Automatisation de la gestion avec PowerShell

2.1 Description de l'objectif

L'objectif de cette partie du projet est d'automatiser la gestion des identités et des accès au sein de l'infrastructure AWS à l'aide de scripts PowerShell.

Cette automatisation vise à simplifier et sécuriser les opérations courantes telles que l'ajout de nouvelles recrues, la gestion des transferts entre vaisseaux, l'administration des groupes fonctionnels et l'application cohérente des politiques de sécurité.

L'utilisation de scripts permet de réduire les erreurs humaines, de gagner du temps et de garantir une application uniforme des règles de sécurité.

2.2 Automatisation de la création des utilisateurs IAM

La création manuelle des utilisateurs IAM lors de l'arrivée de nouvelles recrues ou de transferts est remplacée par un script PowerShell automatisé.

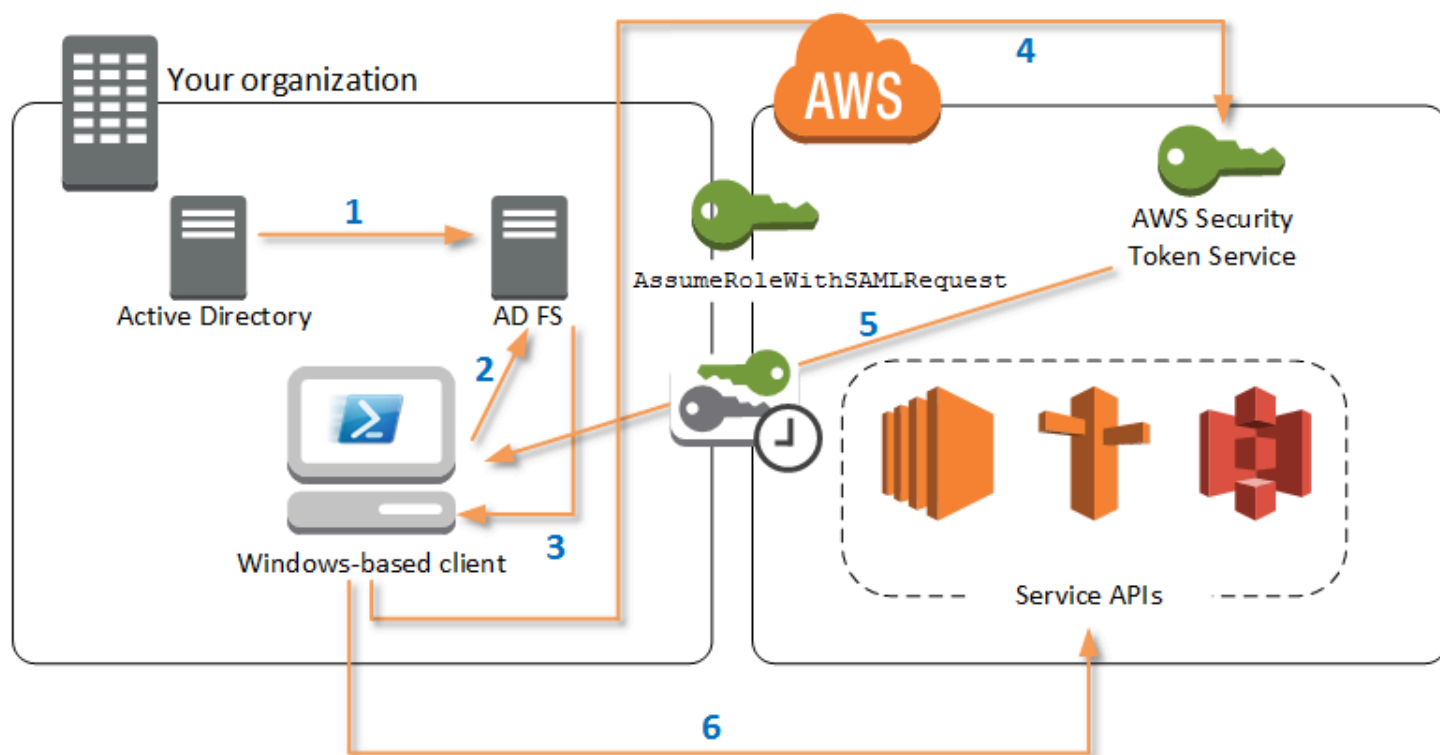


Figure 5 : Automatisation de la création des utilisateurs IAM via PowerShell.

```
# Variables
$UserName = "recrue_spock"
$GroupName = "Exploration"

# Création de l'utilisateur IAM
New-IAMUser -UserName $UserName

# Création d'un accès console (mot de passe initial)
New-IAMLoginProfile `
    -UserName $UserName `
    -Password "MotDePasseTemporaire123!" `
    -PasswordResetRequired $true

# Ajout de l'utilisateur à un groupe
Add-IAMUserToGroup `
    -UserName $UserName `
    -GroupName $GroupName
```


Exemple de mise en situation :

Lorsqu'une nouvelle recrue rejoint Starfleet, ce script est exécuté pour créer automatiquement son compte IAM, lui attribuer un groupe et forcer la modification du mot de passe à la première connexion.

2.3 Gestion automatisée des groupes fonctionnels

Les groupes IAM représentent les équipes opérationnelles (exploration, médical, commandement, etc.).

PowerShell permet d'automatiser l'ajout et la suppression des membres selon leur mission.

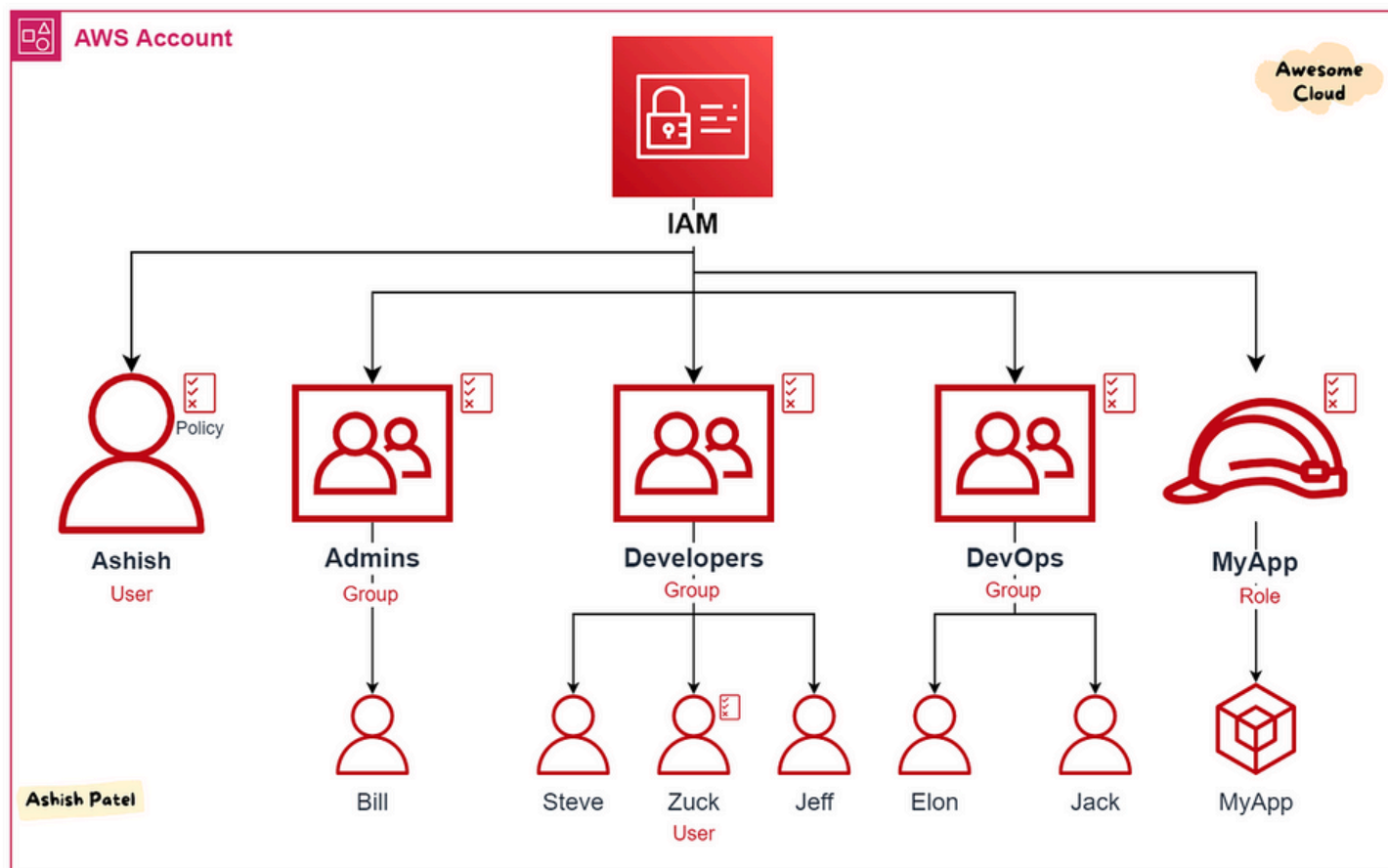


Figure 6 : Gestion des groupes IAM et des permissions associées.

```
# Variables
$UserName = "recrue_spock"
$OldGroup = "Formation"
$NewGroup = "Medical"

# Retrait de l'utilisateur de son ancien groupe
Remove-IAMUserFromGroup `
    -UserName $UserName `
    -GroupName $OldGroup

# Ajout de l'utilisateur au nouveau groupe
Add-IAMUserToGroup `
    -UserName $UserName `
    -GroupName $NewGroup
```

Exemple concret :

Lorsqu'un membre d'équipage est transféré vers une équipe médicale, ses accès sont automatiquement ajustés sans modification manuelle des permissions.

2.4 Application automatique des politiques de sécurité

Pour les missions sensibles, il est essentiel que les politiques de sécurité soient appliquées rapidement et de manière cohérente.

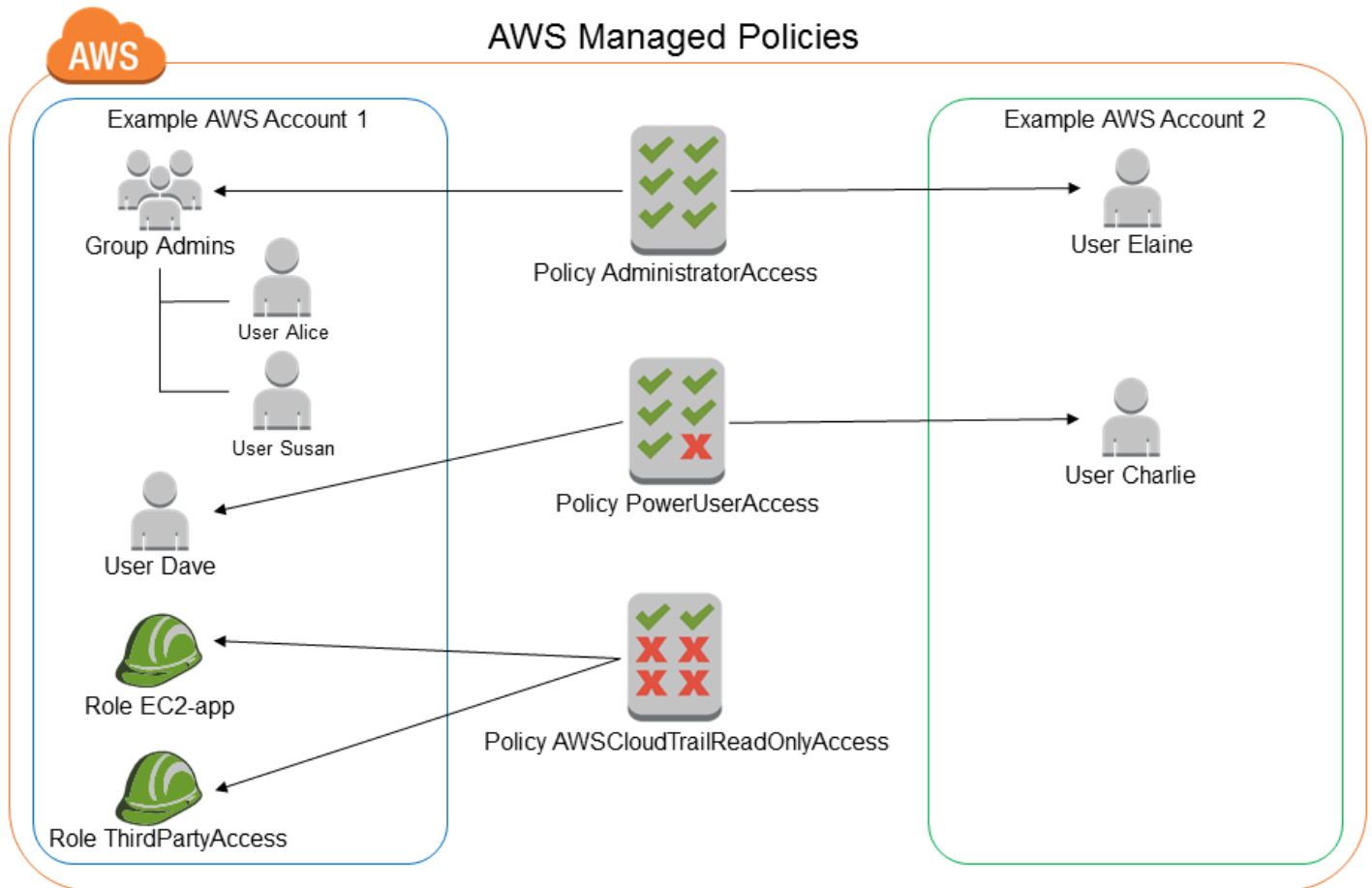


Figure 7 : Application automatique des politiques de sécurité IAM.

```
# Variables
$GroupName = "Exploration"
$PolicyArn = "arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess"

# Attachement d'une politique de sécurité au groupe
Register-IAMGroupPolicy `
  -GroupName $GroupName `
  -PolicyName "MissionSensitivePolicy" `
  -PolicyDocument (Get-Content ".\policy.json" -Raw)

Attach-IAMGroupPolicy `
  -GroupName $GroupName `
  -PolicyArn $PolicyArn
```

Exemple de mise en situation :

Avant une mission sensible, une politique de sécurité restrictive est automatiquement appliquée aux groupes concernés, limitant l'accès aux ressources critiques.

2.5 Révocation automatique des accès après mission

Une fois la mission terminée, les accès doivent être retirés afin de respecter le principe du **moindre privilège**.

```
# Variables
$GroupName = "Exploration"
$PolicyArn = "arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess"

# Détachement de la politique après la mission
Detach-IAMGroupPolicy `
    -GroupName $GroupName `
    -PolicyArn $PolicyArn
```

2.6 Résultats attendus

Grâce à l'automatisation par PowerShell, l'infrastructure AWS bénéficie :

- d'une gestion rapide et standardisée des utilisateurs,
- d'une administration efficace des groupes fonctionnels,
- d'une application cohérente des politiques de sécurité,
- d'une meilleure traçabilité des actions,
- d'un niveau de sécurité renforcé.

2.7 Sécurité et bonnes pratiques des scripts PowerShell

L'automatisation des tâches d'administration implique un niveau de privilège élevé. Il est donc essentiel de sécuriser les scripts PowerShell afin d'éviter les abus, les erreurs critiques ou les compromissions de comptes.

2.7.1 Principe du moindre privilège

Les scripts PowerShell utilisent des identités IAM dédiées, disposant uniquement des permissions nécessaires à l'exécution des actions prévues (création d'utilisateurs, gestion des groupes, attachement de politiques).

Ces identités ne disposent pas de droits administrateur globaux, ce qui limite l'impact en cas de compromission du script ou des identifiants.

2.7.2 Journalisation des actions

Chaque script intègre un mécanisme de journalisation permettant de tracer :

- les actions effectuées,
- la date et l'heure d'exécution,
- l'utilisateur concerné,
- le résultat de l'opération (succès ou échec).

```
$LogFile = "C:\Logs\iam_automation.log"
$TimeStamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"

Add-Content $LogFile "$TimeStamp - Création de l'utilisateur $UserName"
```

Cette journalisation permet de faciliter les audits de sécurité et les investigations en cas d'incident.

2.7.3 Gestion des erreurs et contrôles

Afin d'éviter des incohérences ou des actions non souhaitées, les scripts incluent des mécanismes de gestion des erreurs.

Avant toute action critique, des vérifications sont effectuées pour s'assurer que l'utilisateur ou le groupe n'existe pas déjà, ou que la politique n'est pas déjà appliquée.

```
try {  
    New-IAMUser -UserName $UserName -ErrorAction Stop  
}  
catch {  
    Write-Host "Erreur : l'utilisateur existe déjà ou création impossible."  
}
```

Cette approche permet d'éviter les interruptions brutales et d'améliorer la fiabilité des scripts.

2.7.4 Mode simulation (Dry-Run)

Avant l'exécution réelle des scripts en environnement de production, un mode simulation peut être utilisé afin de vérifier les actions qui seraient effectuées sans appliquer de modifications.

```
$DryRun = $true  
  
if ($DryRun) {  
    Write-Host "Simulation : création de l'utilisateur $UserName"  
} else {  
    New-IAMUser -UserName $UserName  
}
```

Ce mode permet de tester les scripts en toute sécurité et de réduire le risque d'erreurs.

2.7.5 Protection des informations sensibles

Les informations sensibles, telles que les mots de passe ou les clés d'accès AWS, ne sont jamais stockées en clair dans les scripts.

Elles sont récupérées de manière sécurisée via :

- des variables d'environnement,
- des coffres de secrets (ex. AWS Secrets Manager),
- ou des profils AWS configurés localement.

2.7.6 Résumé des mesures de sécurité

Les scripts PowerShell respectent les bonnes pratiques suivantes :

- utilisation d'identités IAM dédiées et limitées,
- journalisation complète des actions,
- gestion des erreurs et contrôles préalables,
- tests en mode simulation,
- protection des informations sensibles.

Objectif 3 : Intégration et sécurisation des applications avec Azure AD (Entra ID)

3.1 Description de l'objectif

Cet objectif vise à intégrer les applications critiques de Starfleet dans Entra ID (Azure AD) afin de centraliser et sécuriser les accès via des mécanismes modernes comme le Single Sign-On (SSO).

L'objectif inclut également l'intégration d'applications personnalisées avec des rôles et permissions spécifiques pour chaque groupe fonctionnel (ex. ingénieurs, officiers de commandement), garantissant ainsi un accès sécurisé et conforme au principe du moindre privilège.

3.2 Intégration des applications SaaS essentielles

Les applications SaaS de Starfleet, telles que le **Journal de Bord (Captain's Log)** et le **Centre de Commandement (Command Center)**, sont intégrées à Entra ID pour permettre :

- un accès centralisé via **SSO**,
- l'utilisation des identifiants Starfleet existants,
- la sécurisation des connexions par MFA ou politiques conditionnelles.

🔧 Exemple – Configuration SSO pour une application SaaS

1. Dans Azure AD : **Ajouter une application d'entreprise**
2. Sélectionner l'application (ex. Captain's Log)
3. Configurer **SSO avec SAML ou OpenID Connect**
4. Définir les utilisateurs ou groupes autorisés

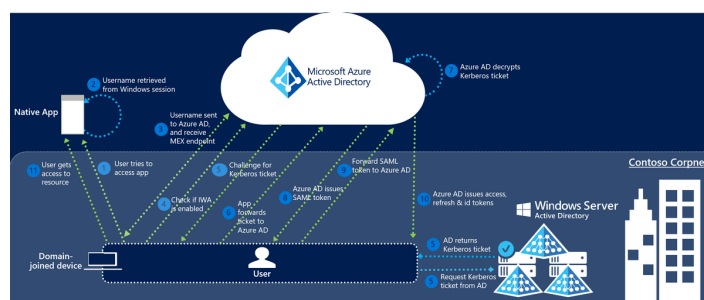
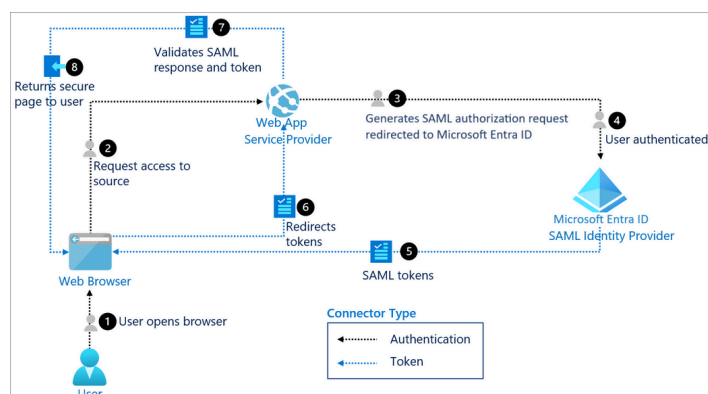


Figure 8 : Flux SSO Azure AD pour l'accès sécurisé aux applications SaaS de Starfleet.

Exemple concret :

Lorsqu'un officier ouvre le Captain's Log, il est automatiquement authentifié via Entra ID, sans avoir besoin de saisir un nouveau mot de passe, tout en respectant les règles MFA définies pour les comptes sensibles.

3.3 Ajout d'une application personnalisée

Pour les besoins spécifiques de l'ingénierie, l'application **Gestion des Réparations (Repair Management)** est ajoutée à Entra ID.

L'intégration comprend :

- l'inscription de l'application dans Azure AD,
- la configuration des rôles et permissions pour les utilisateurs,
- la définition des politiques d'accès conditionnel.

```
# Connexion à Azure AD
Connect-AzureAD

# Création d'un rôle pour les ingénieurs
New-AzureADMSRoleDefinition `
  -DisplayName "EngineerRole" `
  -Description "Peut modifier les données de Repair Management" `
  -RolePermissions @(
    @{ ResourceAppId="RepairAppID"; AllowedResourceActions=@("Write","Read")
  }
)

# Attribution du rôle à un utilisateur
Add-AzureADMSRoleAssignment `
  -PrincipalId "<EngineerUserId>" `
  -RoleDefinitionId "<EngineerRoleId>" `
  -ResourceId "<RepairAppObjectId>"
```

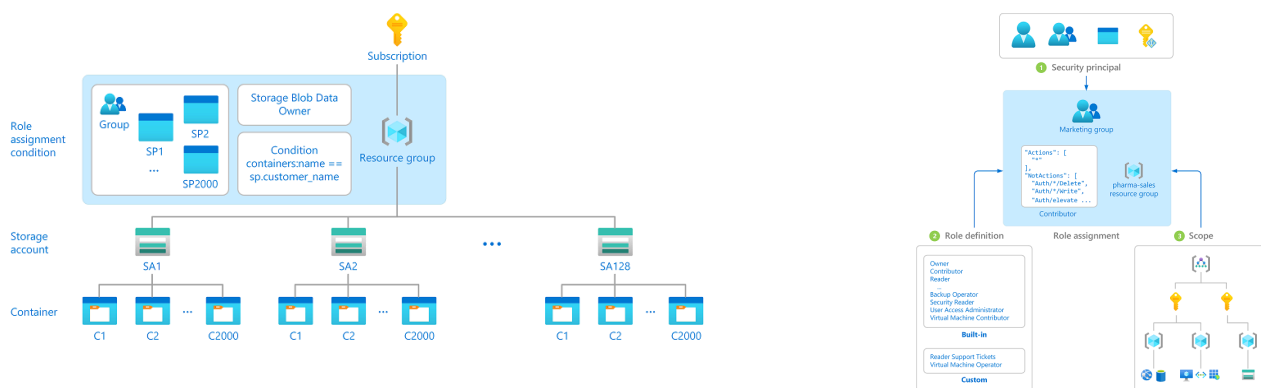


Figure 9 : Attribution de rôles et permissions pour une application personnalisée via Azure AD.

Exemple concret :

Seuls les ingénieurs affectés au groupe **Repair Management** peuvent modifier les données de l'application, tandis que les autres membres de Starfleet ont uniquement un accès en lecture ou sont bloqués.

3.4 Tests et validation des accès

Pour garantir que les rôles et permissions sont correctement appliqués :

1. Tester la connexion des utilisateurs autorisés et vérifier l'accès aux fonctionnalités autorisées.
2. Tester la connexion des utilisateurs non autorisés et confirmer le blocage des actions interdites.
3. Vérifier que le SSO fonctionne correctement et que MFA est appliqué lorsque nécessaire.

Exemple de scénario de test

Utilisateur	Groupe	Action attendue	Résultat réel
Officier supérieur	Command	Accès complet au Command Center	✓ OK
Ingénieur	Repair	Modifier les données Repair	✓ OK
Civil / stagiaire	Aucun	Tentative d'accès	✗ Refusé

Figure 10 : Exemple de test des permissions sur les applications intégrées à Azure AD.

3.5 Résultats attendus

À l'issue de cet objectif :

- Les applications SaaS critiques de Starfleet sont accessibles via **SSO** avec MFA et sécurisation par Entra ID.
- Les applications personnalisées disposent de **rôles et permissions granulaires**, garantissant que chaque utilisateur ne peut effectuer que les actions autorisées.
- Les tests confirment que les politiques de sécurité et l'accès conditionnel fonctionnent correctement.
- L'intégration centralisée réduit les risques de comptes multiples et de fuites de données.

Objectif 4 : Surveillance et réponse aux incidents

4.1 Description de l'objectif

L'objectif de cette partie est de **mettre en place un système de surveillance proactive et de réponse aux incidents** sur l'infrastructure AWS et les applications Starfleet intégrées à Entra ID. Il s'agit de détecter et d'analyser toute activité suspecte, de recevoir des alertes en temps réel, et de tester les procédures de réponse afin de sécuriser les données critiques et les systèmes opérationnels, notamment lors des missions sensibles.

4.2 Surveillance des accès et analyse des logs

La surveillance repose sur l'analyse des logs des systèmes AWS et Azure AD :

- **AWS CloudTrail** pour toutes les actions IAM et les accès aux ressources sensibles (S3, EC2, RDS).
- **AWS GuardDuty** pour détecter des comportements suspects, tels que des connexions depuis des localisations inhabituelles.
- **Azure AD Sign-In Logs** pour les applications SaaS et personnalisées, permettant de suivre toutes les tentatives de connexion.

```
# Connexion à AWS
Import-Module AWSPowerShell
Get-CloudTrailEvent -StartTime (Get-Date).AddDays(-1) -EndTime (Get-Date) |
    Where-Object {$_.EventName -eq "ConsoleLogin" -and $_.ErrorMessage -ne $null}
Select EventTime, Username, SourceIPAddress, ErrorMessage
```

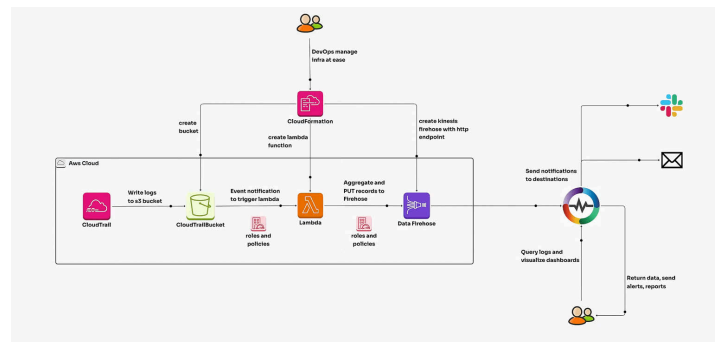
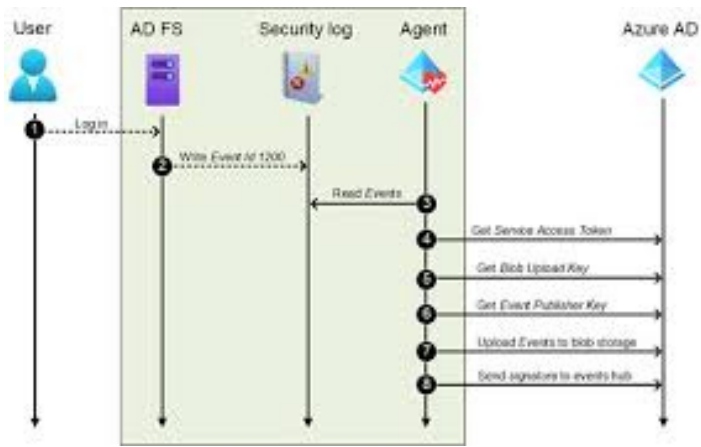


Figure 11 : Surveillance des tentatives de connexion via CloudTrail, GuardDuty et Azure AD logs.

Exemple concret :

Une tentative d'accès non autorisée à la base des plans des moteurs à distorsion est détectée par CloudTrail et GuardDuty. L'événement est enregistré avec l'adresse IP et le compte impliqué.

4.3 Configuration des alertes en temps réel

Pour réagir rapidement aux incidents, des alertes sont configurées via :

- **AWS SNS** pour les événements GuardDuty critiques,
- **Azure AD Alerting** pour les anomalies de connexion sur les applications SaaS et personnalisées.

```
# Définir un topic SNS pour les alertes
$TopicArn = "arn:aws:sns:us-east-1:123456789012:StarfleetAlerts"

# Créer une alerte pour une activité suspecte
New-GDThreatIntelSet -Name "SuspiciousAccess" -Activate $true
Subscribe-SNS -TopicArn $TopicArn -Protocol "email" -Endpoint "security@starfleet"
```

Exemple concret :

Une connexion depuis un secteur de l'espace non autorisé déclenche instantanément une alerte envoyée à l'équipe de sécurité Starfleet.

4.4 Simulation et réponse aux incidents

Afin de tester l'efficacité des procédures de sécurité, des incidents simulés sont réalisés :

- Tentatives de piratage simulées sur les systèmes AWS et applications Entra ID.
- Réinitialisation automatique des accès compromis via scripts PowerShell.
- Mise en quarantaine des systèmes suspects pour éviter la propagation.

```
# Identifier les comptes compromis
$CompromisedUsers = @("recrue_spock","recrue_kirk")

# Réinitialiser les mots de passe IAM
foreach ($User in $CompromisedUsers) {
    Set-IAMUserPassword -UserName $User -Password "MotDePasseTemporaire123!" -Pas
}

# Désactiver les sessions en cours
foreach ($User in $CompromisedUsers) {
    Revoke-IAMUserSession -UserName $User
}
```

```
# Déplacer une instance EC2 suspecte vers un VPC isolé
$InstanceId = "i-0123456789abcdef0"
Stop-EC2Instance -InstanceId $InstanceId
Modify-EC2InstanceAttribute -InstanceId $InstanceId -VpcId "vpc-quarantine"
Start-EC2Instance -InstanceId $InstanceId
```

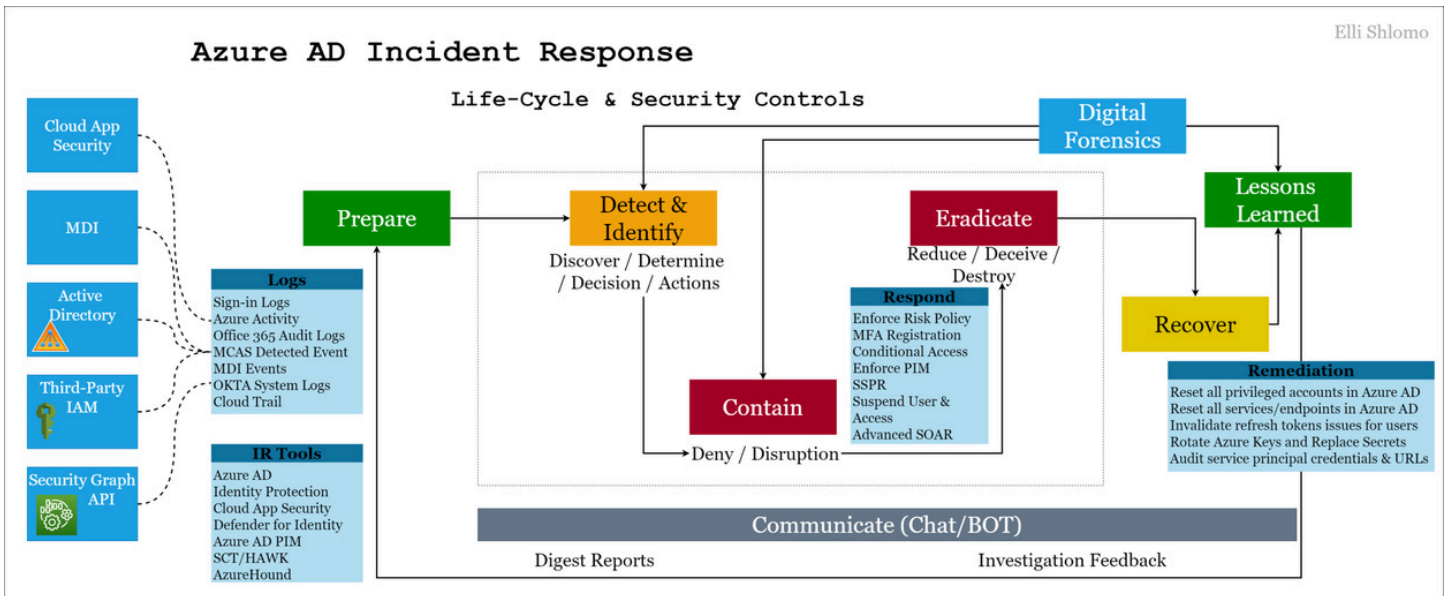
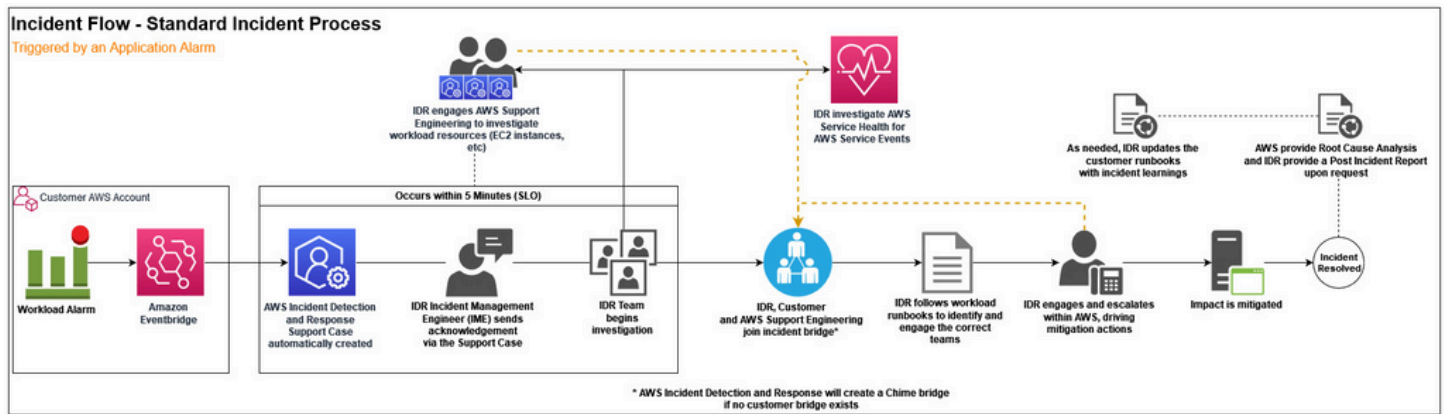



Figure 12 : Exemple de procédure de réponse aux incidents pour les systèmes AWS et Azure AD.

Exemple concret :

Lors d'un test de piratage simulé, le système détecte une intrusion, réinitialise les comptes compromis et met les instances suspectes en quarantaine, assurant la continuité des opérations Starfleet.

4.5 Résultats attendus

À l'issue de cet objectif :

- Les tentatives d'accès non autorisées sont détectées en temps réel.
- Les incidents sont analysés et tracés via les logs CloudTrail et Azure AD.
- Les alertes sont transmises immédiatement à l'équipe sécurité.
- Les procédures de réponse automatisées garantissent la protection des ressources critiques et la reprise rapide des opérations.
- L'infrastructure Starfleet dispose d'un **cycle complet de surveillance et de réaction proactive**.